

Recurso de profundización – Clase 4

Título del recurso relacionado:

Del asincronismo al control de errores: simulando un servicio confiable en JavaScript moderno

Descripción del recurso relacionado:

Este recurso permite experimentar cómo JavaScript gestiona operaciones asíncronas (como solicitudes a un servidor) y cómo aplicar un manejo de errores robusto usando `async/await`, `try...catch` y clases personalizadas de error.

A través de un mini laboratorio, el estudiante construirá una simulación de un servicio remoto, con respuestas aleatorias (éxito, error o demora), para comprender cómo funcionan las promesas y cómo mantener el control total del flujo del programa incluso ante fallas.

Objetivo del recurso:

Comprender cómo las promesas y el manejo estructurado de errores (mediante `try...catch` y `throw`) permiten desarrollar aplicaciones web más seguras, predecibles y resilientes ante fallas de red, datos o validaciones.

Desarrollo paso a paso

Paso 1. Crear el entorno base

Crea un nuevo archivo `simulador.html` con la siguiente estructura mínima:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Simulador de servicio asíncrono</title>
</head>
<body>
  <h1>Simulador de servicio remoto con Promesas y errores</h1>
  <button id="btn-ok">Simular éxito</button>
  <button id="btn-error">Simular error</button>
  <button id="btn-random">Simular aleatorio</button>

  <p id="estado">Esperando acción...</p>
  <pre id="log"></pre>

  <script src="simulador.js"></script>
</body>
</html>
```

Este documento genera tres botones que permitirán probar distintos resultados de una operación asíncrona simulada.

Paso 2. Simular una operación remota con Promesas

Crea un archivo `simulador.js` en el mismo directorio:

```
// Función que simula una operación asíncrona
function servicioSimulado(forzarError = false) {
  return new Promise((resolve, reject) => {
    const tiempo = Math.floor(Math.random() * 2000) + 1000; // 1-3s
    const exito = forzarError ? false : Math.random() > 0.3; // 70% éxito

    setTimeout(() => {
      if (exito) {
        resolve({ mensaje: "Datos recibidos correctamente", codigo: 200 });
      } else {
        reject({ mensaje: "Falla en el servidor remoto", codigo: 500 });
      }
    }, tiempo);
  });
}
```

Conceptos clave:

- Se usa `Promise` para encapsular la operación.
- `resolve()` representa una respuesta exitosa.
- `reject()` simula un error del servidor.
- `setTimeout` emula la latencia de red.

Paso 3. Manejar la Promesa con `async/await` y `try...catch`

Agrega lo siguiente al final del archivo:

```
// Elementos de la interfaz
const estado = document.getElementById("estado");
const log = document.getElementById("log");

// Función genérica para ejecutar el servicio
async function ejecutarSimulacion(forzarError = null) {
  estado.textContent = "Procesando...";
  log.textContent = "";

  try {
    // Se lanza la promesa y se espera su resultado
    const respuesta = await servicioSimulado(forzarError);
    estado.textContent = "✅ Éxito";
    log.textContent = JSON.stringify(respuesta, null, 2);
  } catch (error) {
    // Se captura cualquier error (objeto rechazado)
    estado.textContent = "❌ Error detectado";
    log.textContent = JSON.stringify(error, null, 2);
  } finally {
    // Se ejecuta siempre, ocurra o no error
    console.log("Simulación finalizada");
  }
}

// Enlaces a los botones
document.getElementById("btn-ok").onclick = () => ejecutarSimulacion(false);
document.getElementById("btn-error").onclick = () => ejecutarSimulacion(true);
document.getElementById("btn-random").onclick = () => ejecutarSimulacion(null);
```

Paso 4. Extender con una clase de error personalizada

Añade esta clase antes de la función `ejecutarSimulacion`:

```
// Clase de error personalizada
class ServicioError extends Error {
  constructor(mensaje, codigo) {
    super(mensaje);
    this.name = "ServicioError";
    this.codigo = codigo;
  }
}
```

Y modifica el `catch` dentro de `ejecutarSimulacion` para usarla:Resultado esperado

Cada vez que presiones un botón:

- “Simular éxito” mostrará un mensaje de confirmación.
- “Simular error” devolverá un error controlado con código 500.
- “Simular aleatorio” alternará entre ambos resultados según un 70%/30%.

Tabla 1. Comparación entre `resolve`, `reject` y `throw`

Acción	Contexto	Resultado	Ejemplo
<code>resolve()</code>	Dentro de una Promesa	Entrega datos exitosos	<code>resolve({ok:true})</code>
<code>reject()</code>	Dentro de una Promesa	Rechaza la operación	<code>reject({error:'Falló'})</code>
<code>throw</code>	Dentro de un bloque <code>try</code> o función <code>async</code>	Lanza error capturable	<code>throw new Error('Sin conexión')</code>

Ejercicio complementario

Modifica el simulador para que:

- Cuando el error sea 500, el sistema intente reintentar la solicitud una vez más.
- Si vuelve a fallar, muestre un mensaje:
"Reintento fallido. Verifica tu conexión o inténtalo más tarde."

Pista: puedes envolver `await servicioSimulado()` en otro `try...catch` dentro del bloque principal.

Conclusión del recurso

Este ejercicio demuestra cómo **las Promesas y `async/await` permiten modelar la realidad de la red**, donde los errores son normales, y cómo un buen diseño asíncrono puede mantener la aplicación estable, informando al usuario sin bloquear la ejecución. Con estas prácticas, el estudiante comprende que la robustez de una aplicación depende más de cómo maneja sus fallas que de cuántas tiene.